Local ML in Rails: Fast, Cheap, and Predictable

Running a Japanese language model inside a Rails app

Ryan Carroll - May 2026

What we’re going to look at

1. Why local NLP in Rails?
2. What I built
3. How it works
4. The model and pipeline
5. What you can do

Why did I even do this?

LLMs are pretty great… but

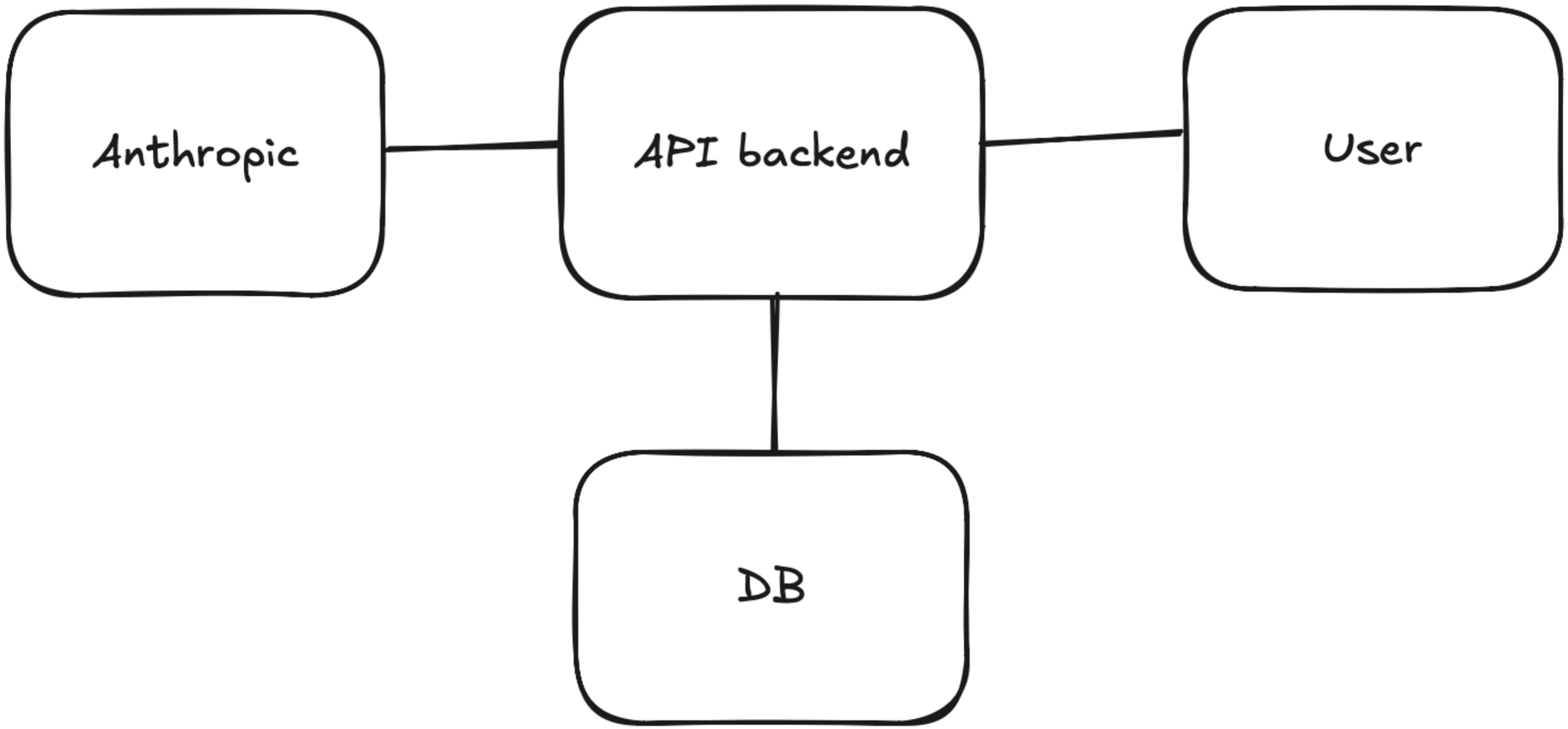
- Expensive to use at scale
- Heavy to deploy and operate
- Latent
- Unpredictable (unless you have a lot of 💸💸💸)

Local ML is the new hotness

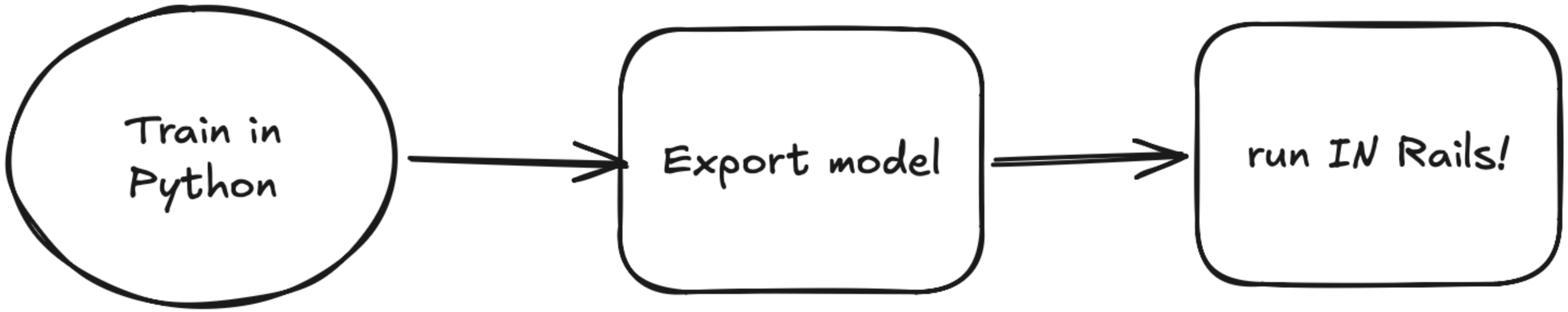
- Focused
- Predictable
- Faster
- Cheaper
- Privacy-forward



Look familiar?



What if…



Train anywhere, deploy anywhere

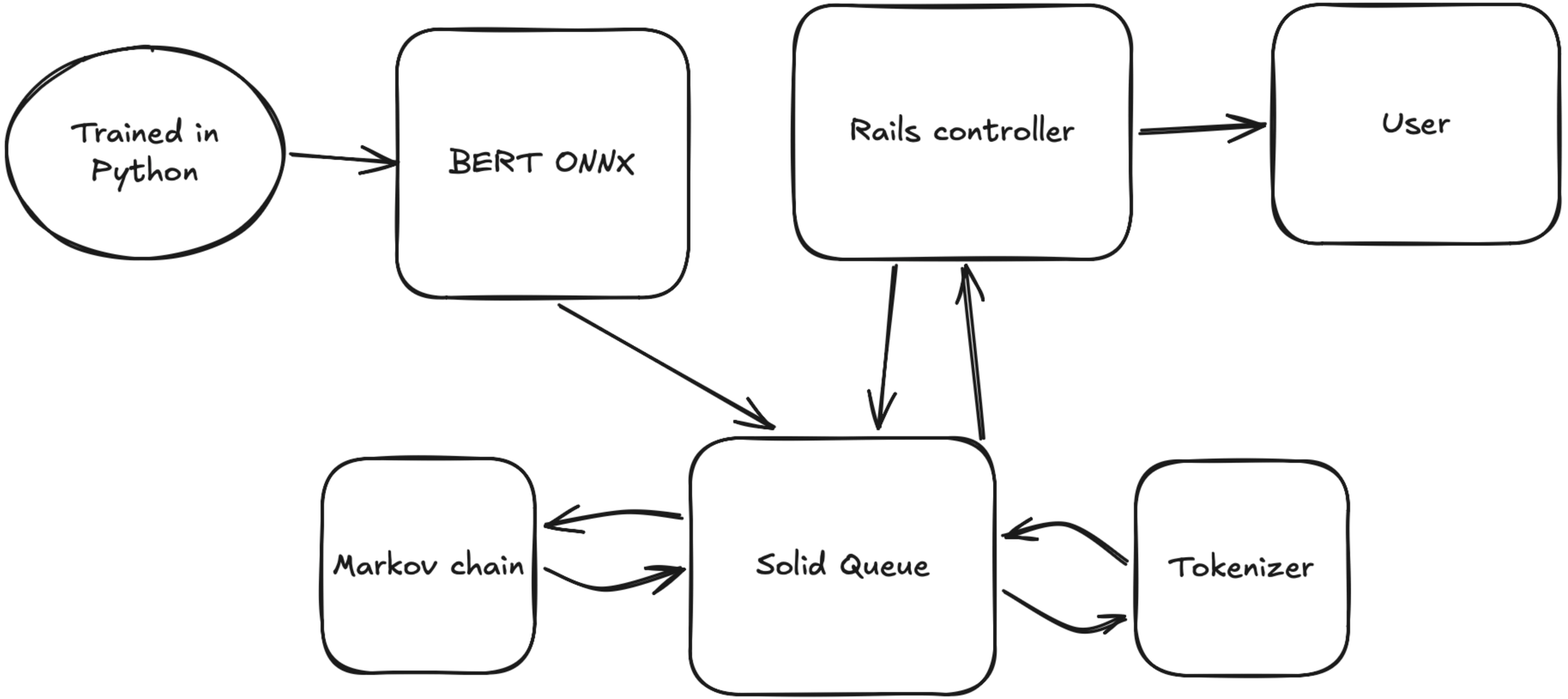
What did I build?

- Sentiment Omikuji
- Japanese text > sentiment > fortune
- Inference done in Rails



日本語はちょっと難しい

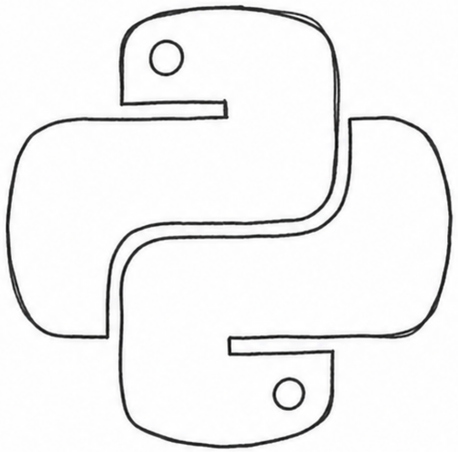
Architecture



Pieces and Parts

The sentiment model

- Fine-tuned in Python
- Exported to ONNX runtime
- BERT model
- Bidirectional Encoder Representations from Transformers



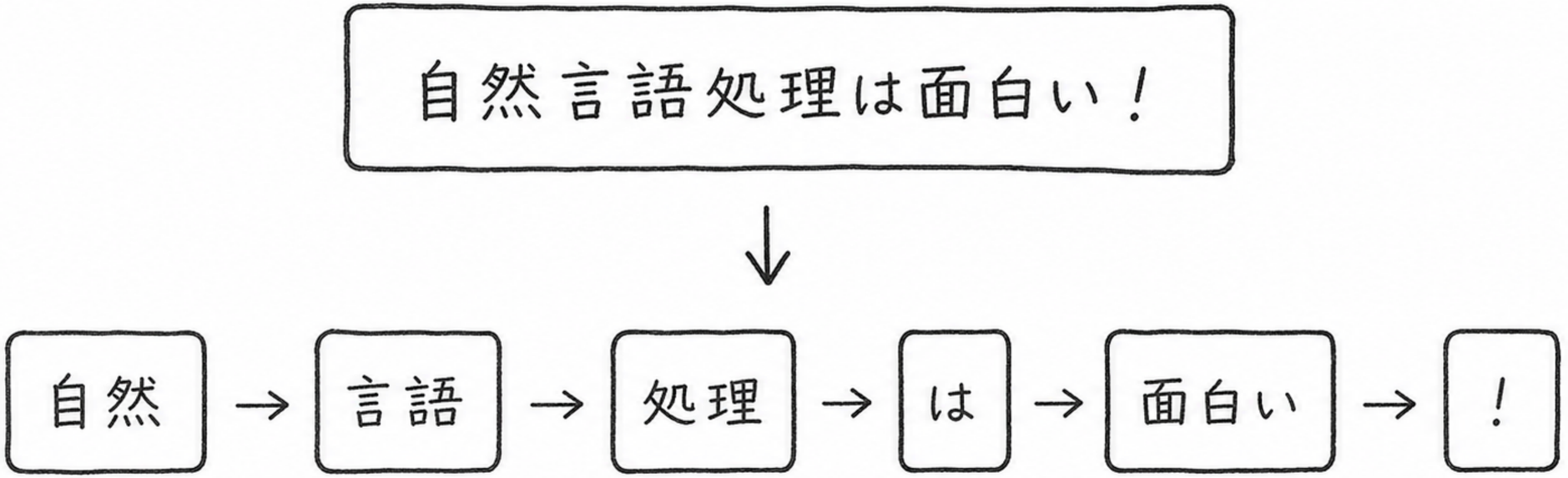
Pieces and Parts

Tokenizer

- Language models- tokens in, tokens out
- Implemented in Ruby



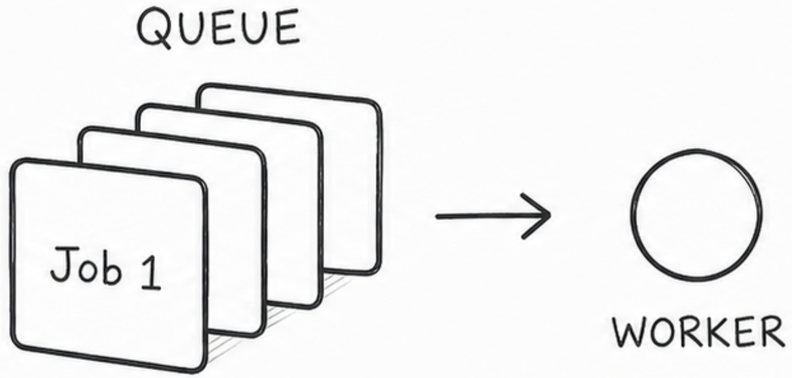
トークナイズ（Tokenize）



Pieces and Parts

SolidQueue

- Orchestrates all the things
- The inference layer
- Frees up the web server
- Scales with the app



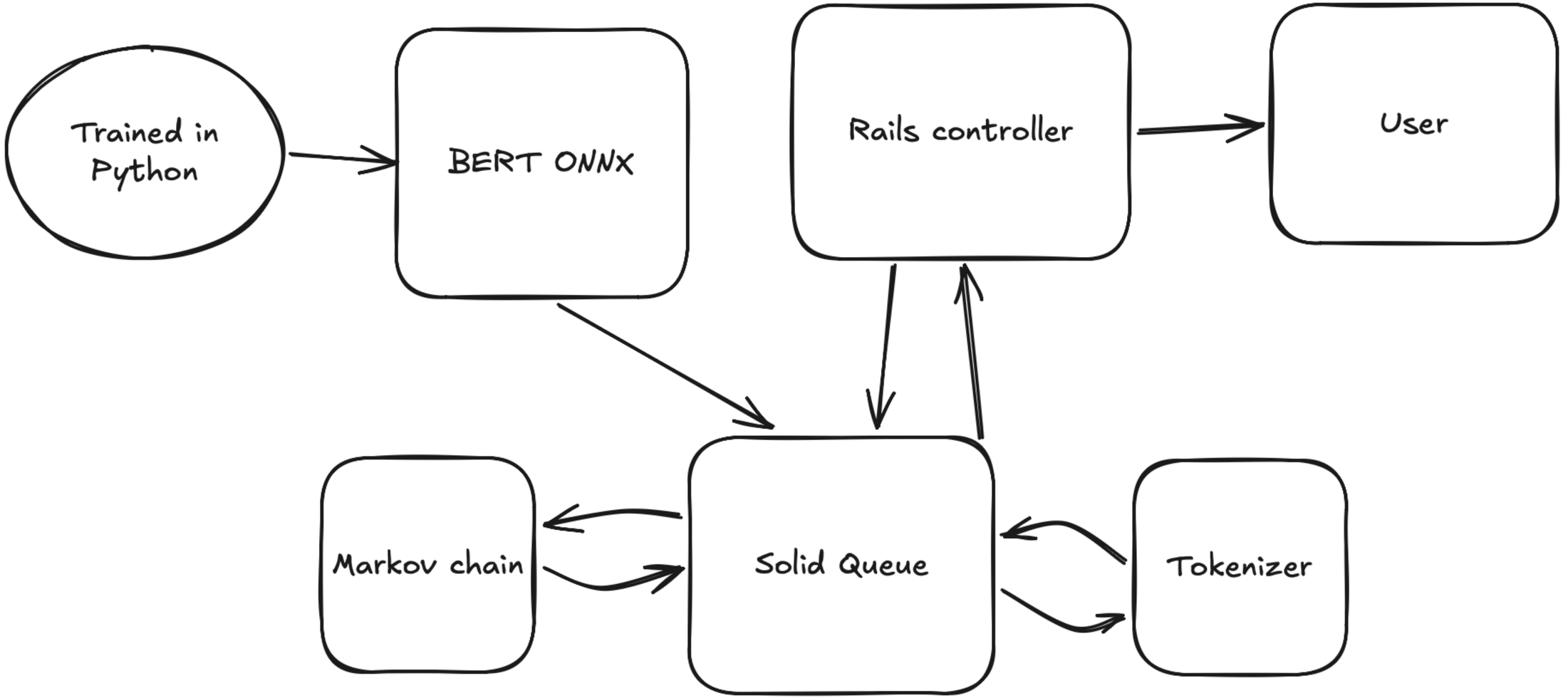
Markov chain

Non-LLM generation

- Simple, fast, easy to run in Ruby
- Gives the app a “voice”
- Style > deep reasoning
- Not limited to text!



Putting it all together



So, what can you do?

(This is the part where you feel inspired)

- Moderation and safety checks
- Messaging-based triage
- Text adventure \o/
- Sentiment-aware recommendation engine
- Sentiment/emotional drift detection

Not every feature needs deep reasoning

Some features just need structure

Some just need a classifier

Some just need a little voice

___ can do that locally

What are you going to build?

AI doesn’t have to mean an LLM

